IT1170 – Data Structures and Algorithms

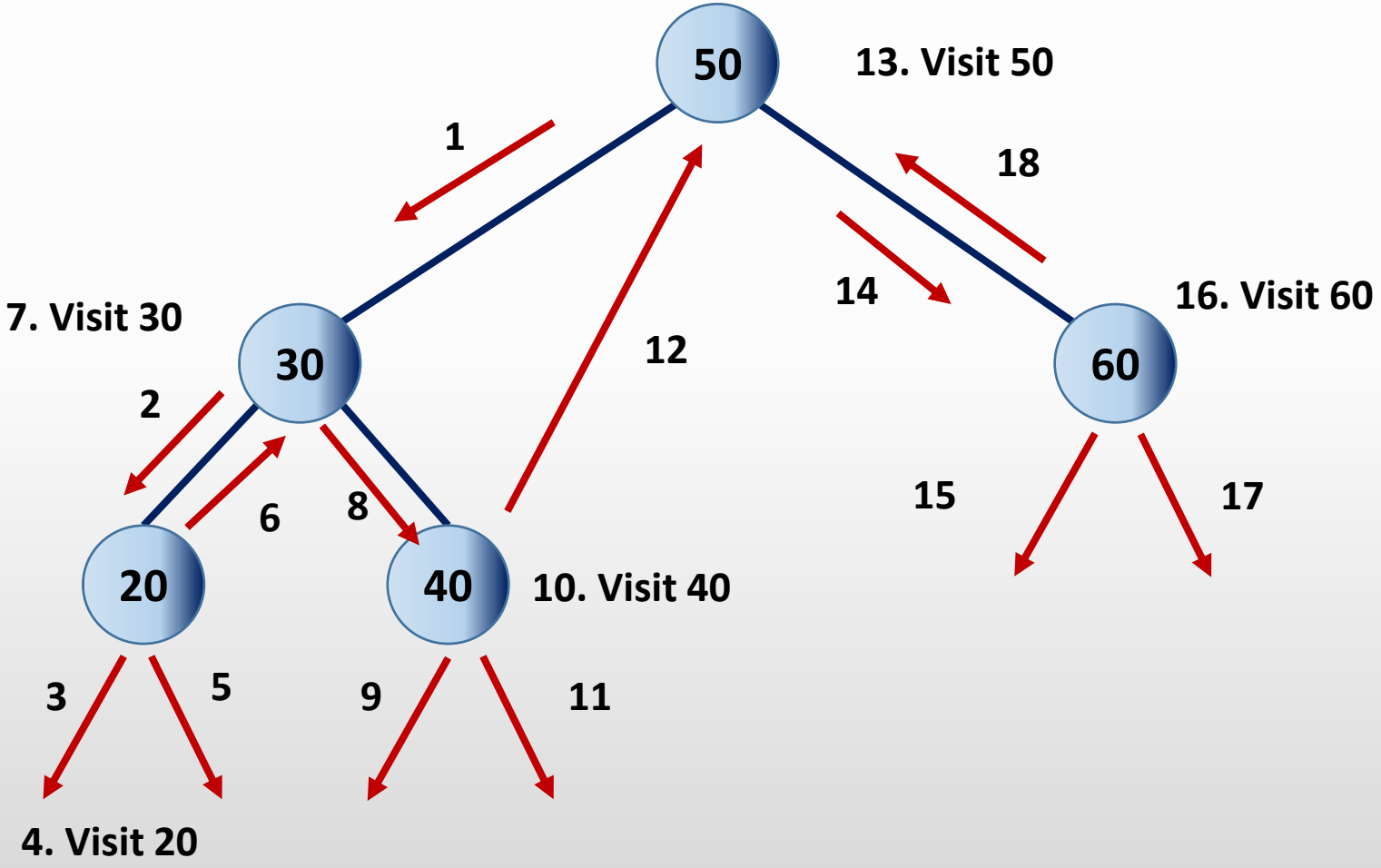
Lecture 07

Trees - 2

Inorder traversing

- Call itself to traverse the node's left subtree
- Visit the node
- Call itself to traverse the node's right subtree

Inorder traversing cont…

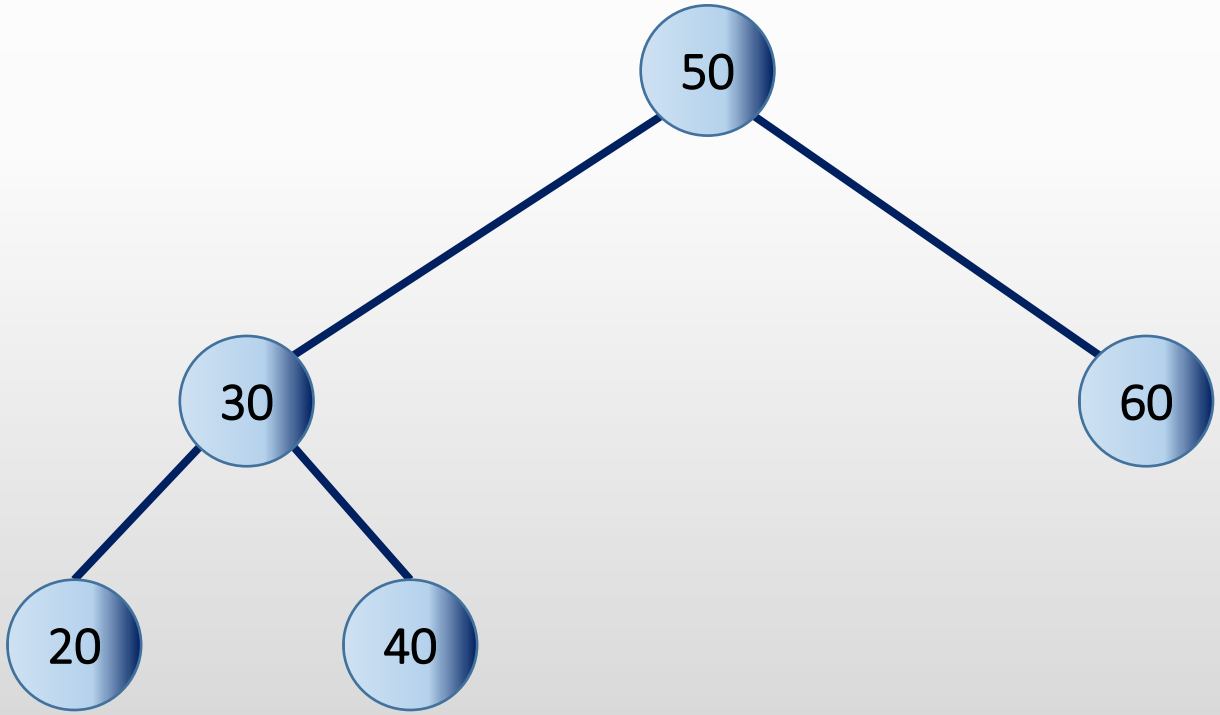


Preorder traversing

- Visit the node
- Call itself to traverse the node's left subtree
- Call itself to traverse the node's right subtree

Question 2

- Write the output, if the following tree is traverse in preorder.

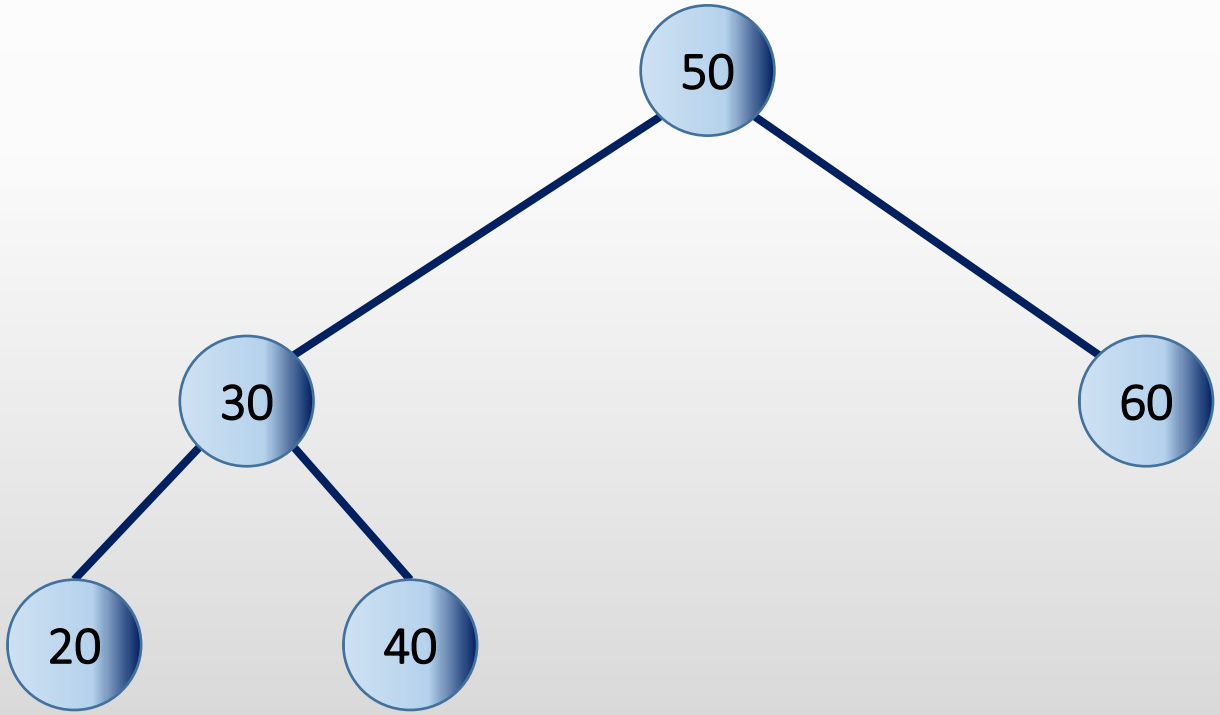


Postorder traversing

- Call itself to traverse the node's left subtree
- Call itself to traverse the node's right subtree
- Visit the node

Question 3

- Write the output, if the following tree is traverse in postorder.



Binary search tree - Implementation

Node Class

Node contains the information ab
an object.
Each node should have
a key, data and
reference to left and
right child.

```
class Node
{
    public int iData; // data item (used as key value)
    public double dData; //other data
    public Node leftChild; // this node's left child
    public Node rightChild; //this node's right child

    public void displayNode( ){
        System.out.print("{");
        System.out.print(iData);
        System.out.print(", ");
        System.out.print(dData);
        System.out.print( " } ");
    }
}
```

Binary search tree - Implementation

Tree Class

```
class Tree
{
        private Node root; // first node of tree

        public Tree(){
                root = null;
        }
        public void insert (int id, double dd){
        }
        public boolean delete (int id){
        }
        public Node find (int key){
        }
    }
```

Binary search tree - Implementation

Tree Class – Find Method

```
class Tree
{
        private Node root;
        public Tree(){
            root = null;
        }
        public void insert (int id, double dd){
        }
        public void delete (int id){
        }
        public Node find(int key){
            Node current = root;
            while (current.iData != key)
            {
                if(key < current.iData)
                  current = current.leftChild;
                else
                  current = current.rightChild;
                if (current == null)
                  return null;
            }
            return current;
        }
```

Binary search tree - Implementation

```
class Tree{
    private Node root;
    ..........
    ..........
    public void insert ( int id , double dd){
            Node newNode = new Node();
            newNode.iData = id;
            newNode.dData = dd;
            if (root == null) // no node in root
                root = newNode;
            else            // root occupied
            {
                Node current = root; //start at root
                Node parent;
                while (true)
                {
                    parent = current;
```

```
                    if (id < current.iData) // go left
                    {
                        current = current.leftChild;
                        if (current == null) {
                            parent.leftChild = newNode;
                            return;
                        }
                    }
                    else // go right
                    {
                        current = current.rightChild;
                        if (current == null){
                            parent.rightChild = newNode;
                            return;
                        }
                    }
                }
            }
        }
```

Binary search tree - Implementation

Tree Class – Inorder Traversing Method

```
private void inOrder(Node localRoot)
{
        if (localRoot != null)
         {
                inOrder(localRoot.leftChild);
                localRoot.displayNode();
                inOrder(localRoot.rightChild);
        }
}
```

Binary search tree - Implementation

Tree Class – Preorder Traversing Method

```
private void preOrder(Node localRoot)
{
        if (localRoot != null)
        {
                localRoot.displayNode();
                preOrder(localRoot.leftChild);
                preorder(localRoot.rightChild);
        }
}
```

Binary search tree - Implementation

Tree Class – Postorder Traversing Method

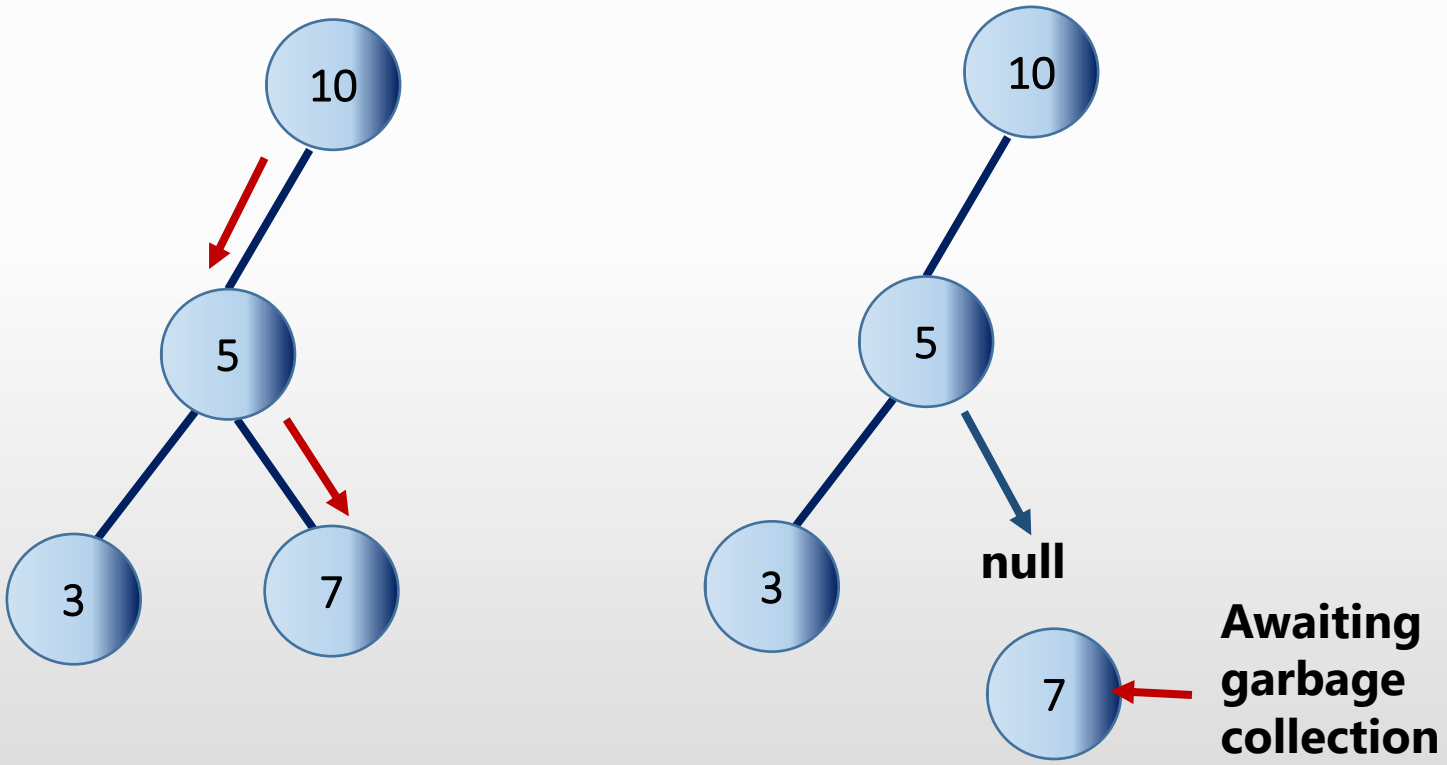
```
private void postOrder(Node localRoot)
{
        if (localRoot != null)
        {
                postOrder(localRoot.leftChild);
                postOrder(localRoot.rightChild);
                localRoot.displayNode();
        }
}
```

Operations - Delete

- First find the node to be deleted.
- If the node to be deleted is found there are three cases to be considered. Whether
  - The node to be deleted is a leaf
  - The node to be deleted has one child
  - The node to be deleted has two children.

Case 1 : The node to be deleted has no children

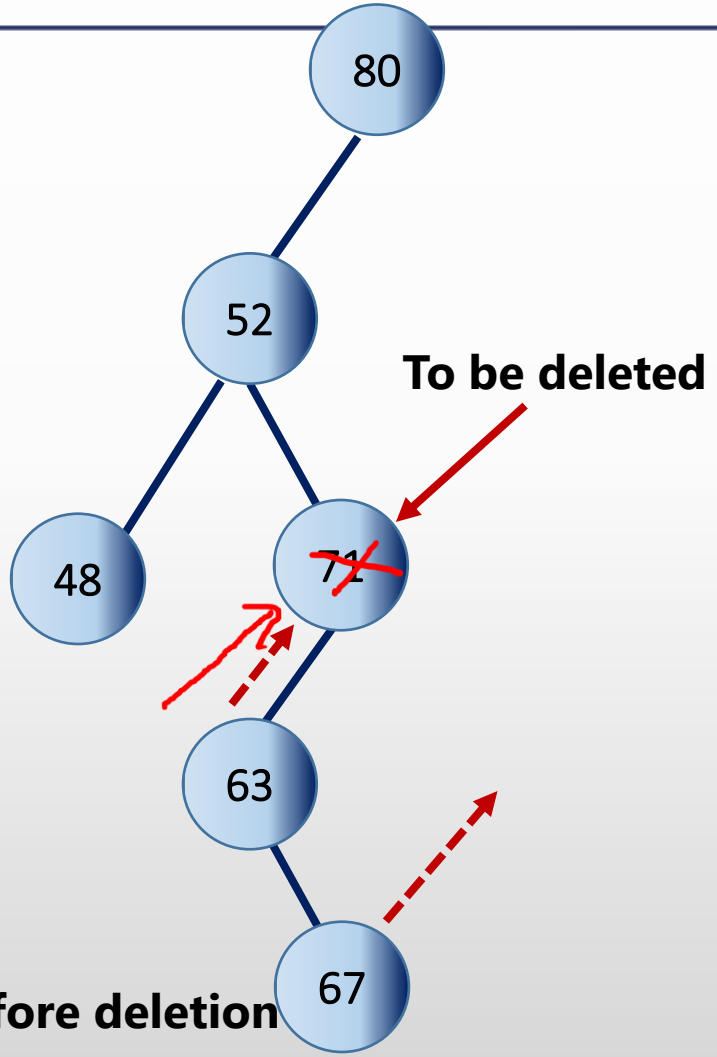
Delete 7



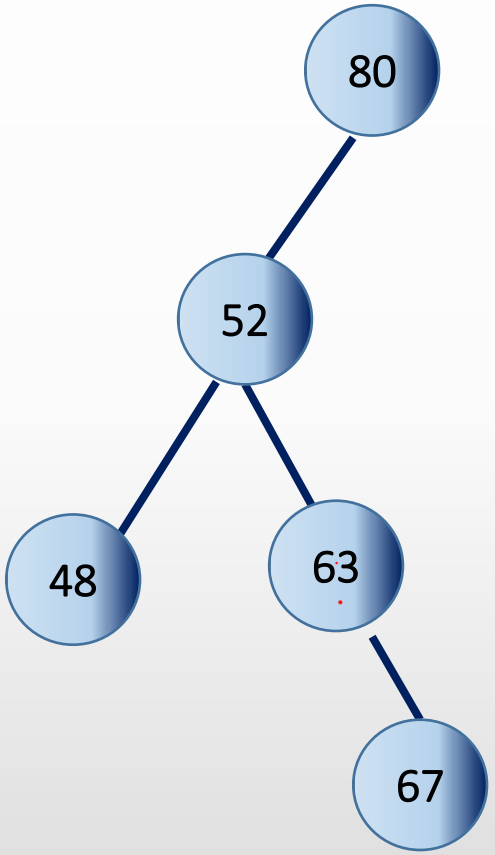
a) Before deletion

b) After deletion

Case 2 : The node to be deleted has one child

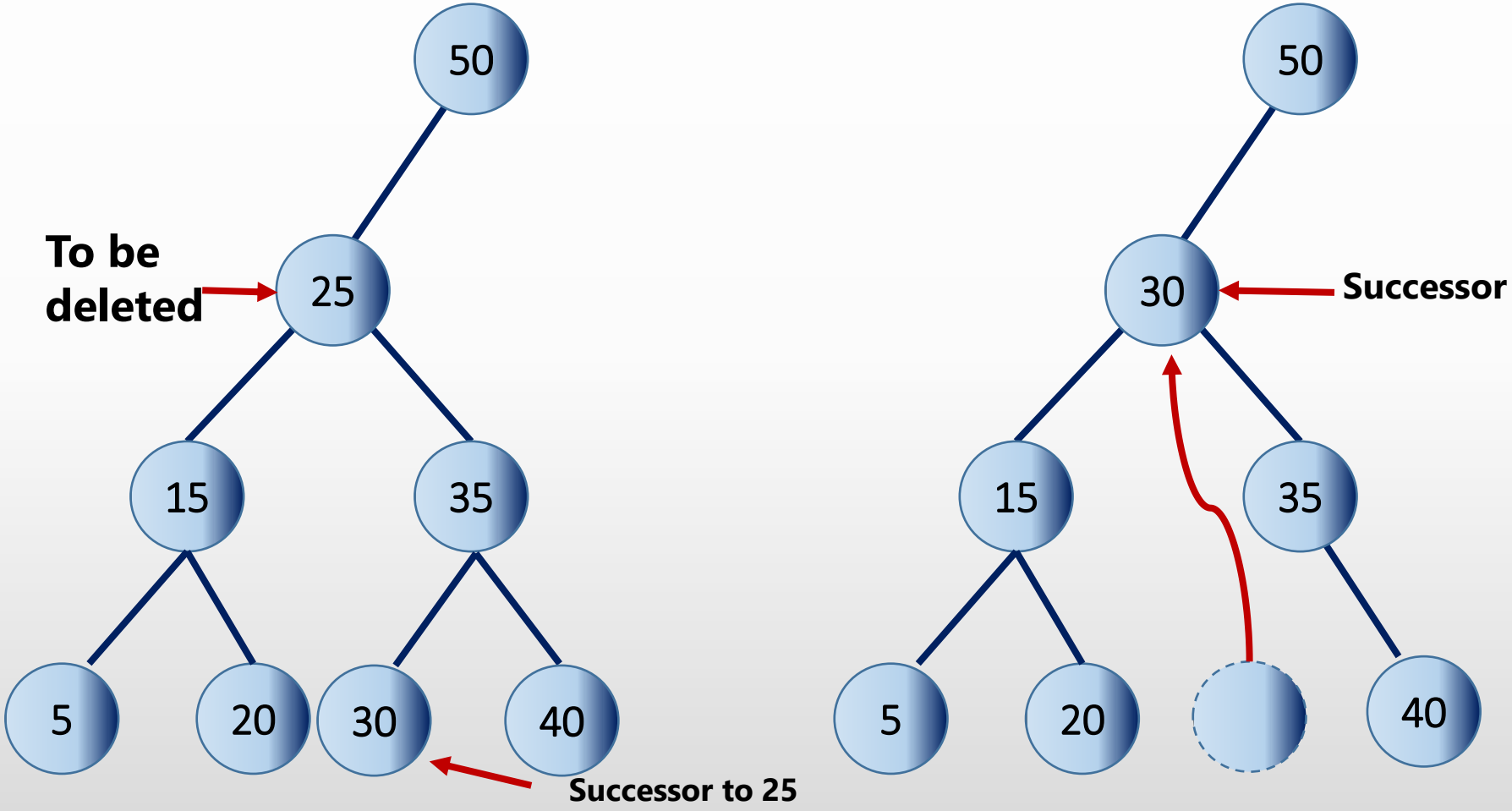


a) Before deletion



b) After deletion

Case 3 : The node to be deleted has two children

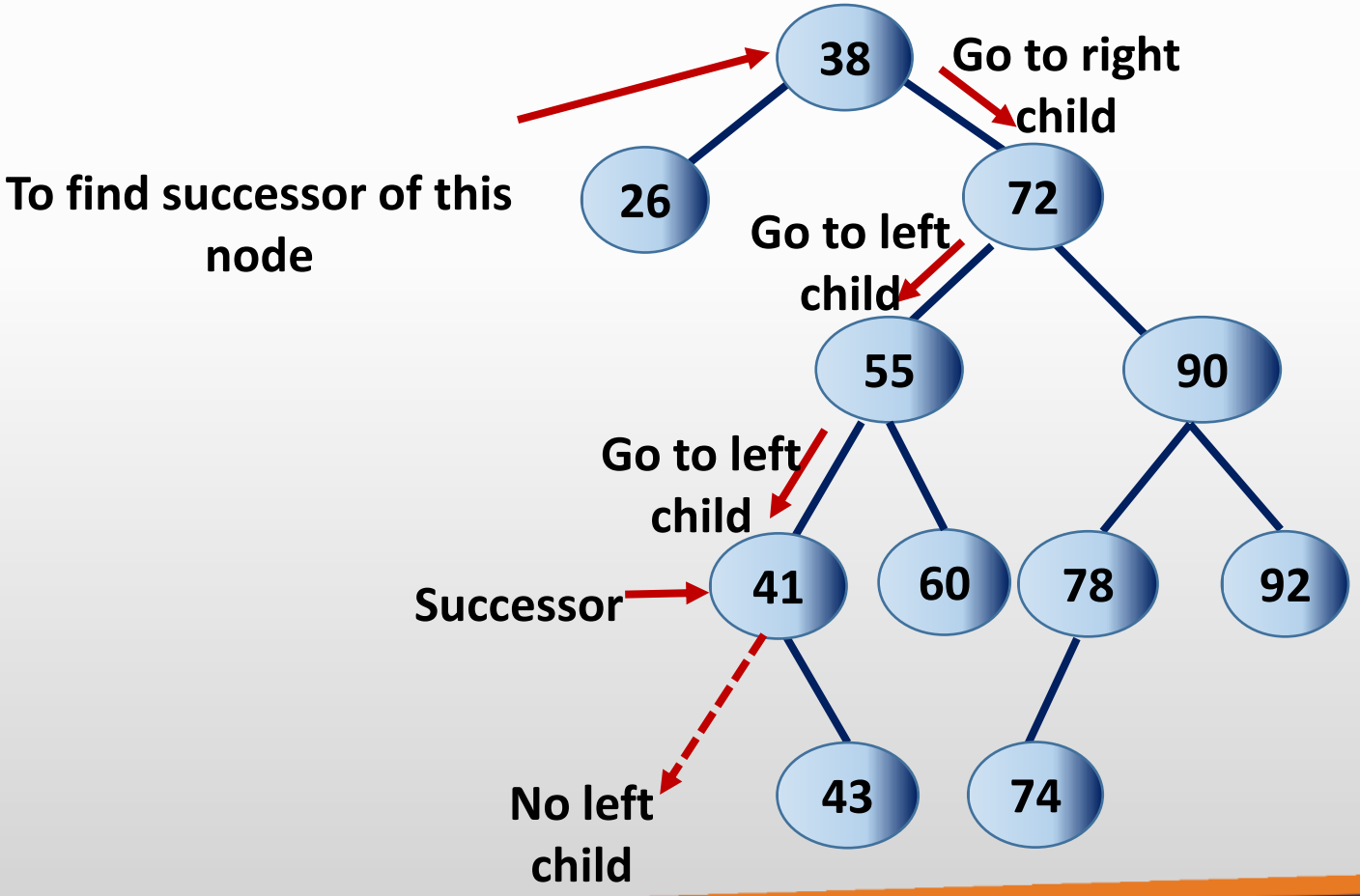


a) Before deletion

b) After deletion

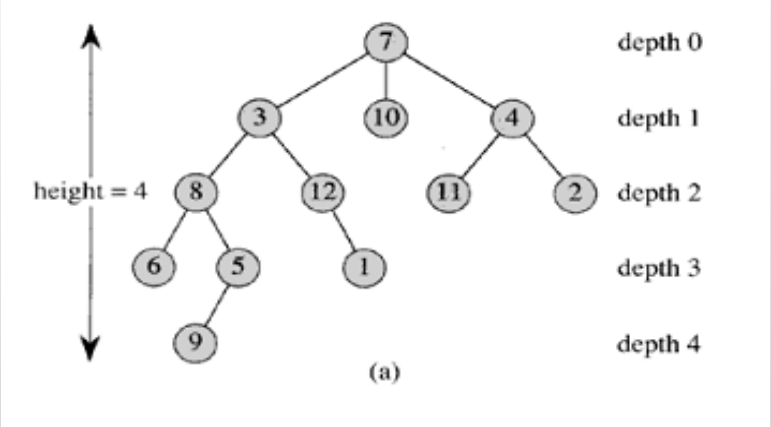
How to find a successor of a node?

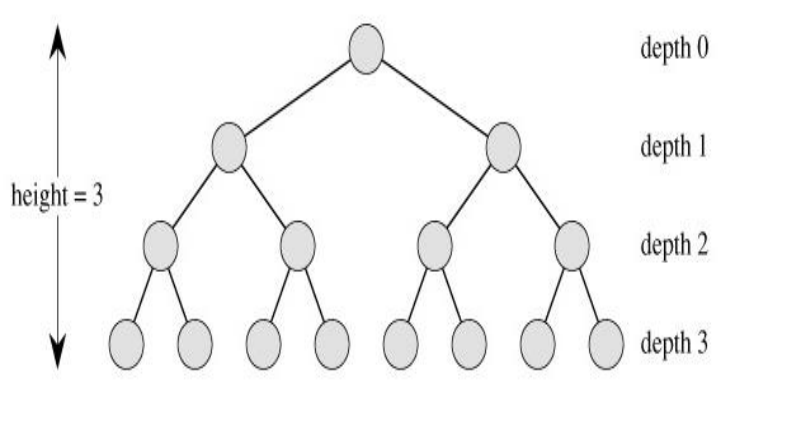
- In a Binary Search Tree, successor of a node is a node with next-highest key.



Tree Terminology

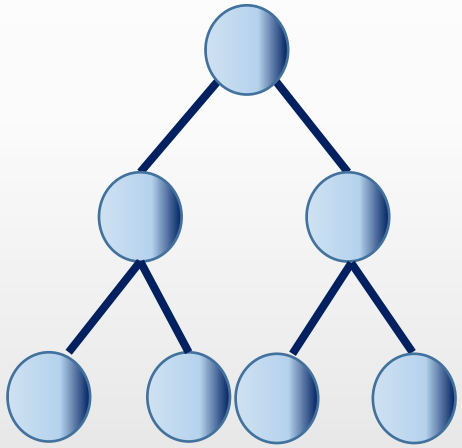
- **Degree of a node:** The number of children it has
- **Depth**: The depth of x in a tree is the length of the path from the root to a node x.
- **Height**: The largest depth of any node in a tree.





Full Binary Tree

- A Full binary tree of height ***h*** that contains exactly $\mathbf{2^{h+1}-1}$ nodes



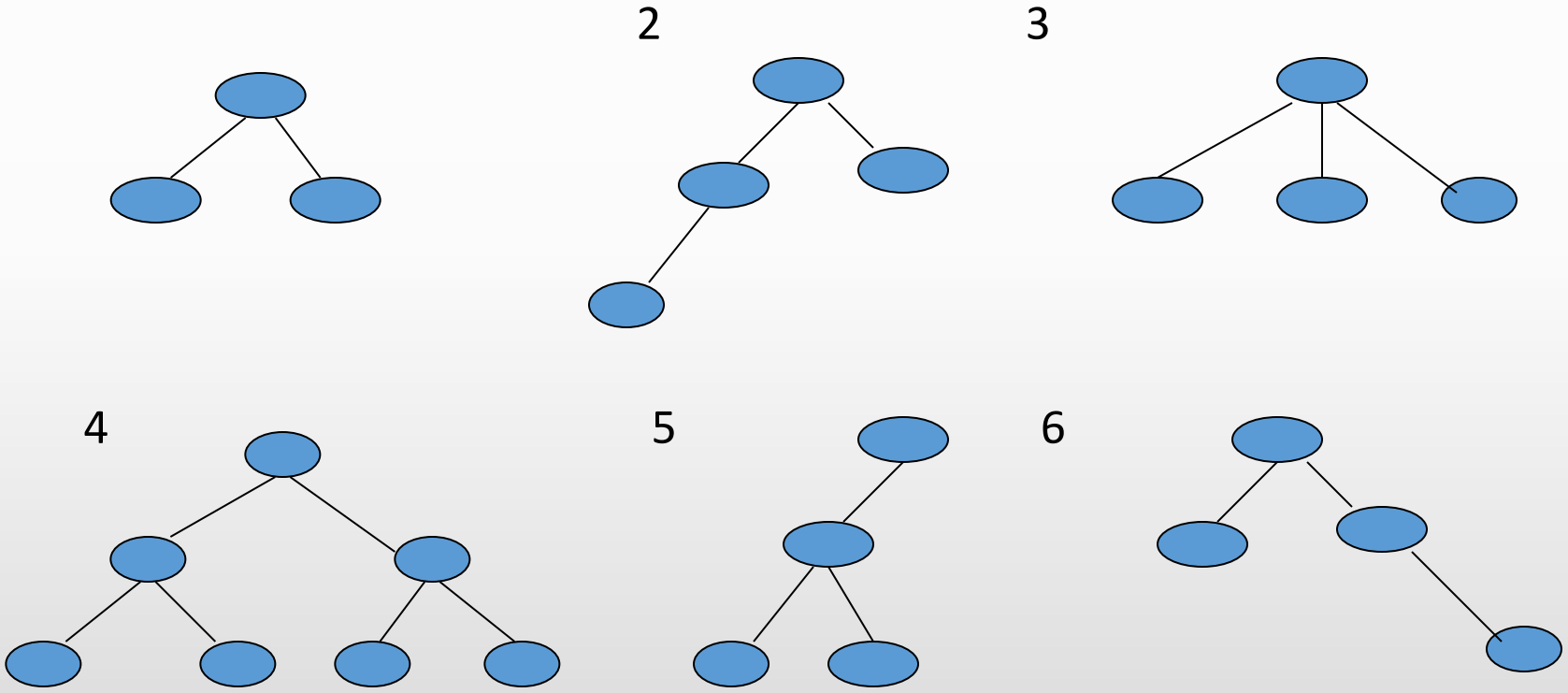
Height, ***h*** = 2, nodes = $2^{2+1}-1 = 7$

Complete Binary Tree

- It is a Binary tree where each node is either a leaf or has degree $\leq$ **2**.
- Completely filled, except possibly for the bottom level
- Each level is filled from **left to right**
- All nodes at the lowest level are as far to the left as possible
- Full binary tree is also a complete binary tree

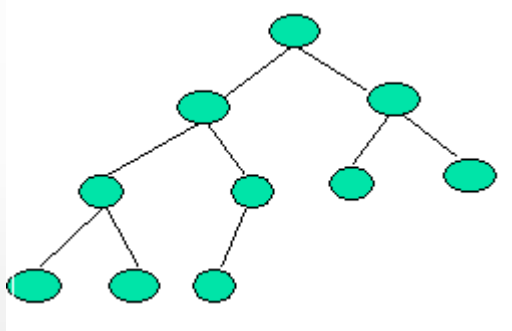
Question

- Find Complete Binary Trees



Height of a complete binary tree

- Height of a complete binary tree that contains ***n*** elements is $\lfloor \log_2(n) \rfloor$
- Example



Above is a Complete Binary Tree with height = 3

No of nodes: n = 10

Height = $\lfloor \log_2(n) \rfloor = \lfloor \log_2(10) \rfloor = 3$